```
!pip install -q sentence-transformers pandas scikit-learn matplotlib seaborn
```

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    accuracy_score, f1_score, precision_score, recall_score,
    classification_report, confusion_matrix
)
import matplotlib.pyplot as plt
import seaborn as sns
from sentence_transformers import SentenceTransformer
import warnings
warnings.filterwarnings('ignore')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu') # making sure gpu (but hardly necessary for colab ig)
print(f"Using device: {device}")
```

```
Using device: cuda
```

```
# DialFRED human QA CSV directly from git
!wget -q https://raw.githubusercontent.com/xfgao/DialFRED/main/data/dialfred_human_qa.csv -O dialfred_human_qa.csv
print("Downloaded DialFRED dataset.")
```

```
Downloaded DialFRED dataset.
```

```
# Column names based on DialFRED documentation
column_names = [
    'split',           # 1  - train / valid_seen / valid_unseen
    'task_id',         # 2  - Task ID
    'trial_id',        # 3  - Trial ID
    'room_type',       # 4  - Room type
    'task_type',       # 5  - Task type from ALFRED
    'subgoal_start',   # 6  - Subgoal start time
    'subgoal_end',     # 7  - Subgoal end time
    'num_actions',     # 8  - Number of low-level actions
    'subgoal_idx',     # 9  - Sub-goal index
    'instruction',     # 10 - Instruction text
    'verb',            # 11 - Low-level actions in sub-goal
    'noun1',           # 12 - First noun
    'noun2',           # 13 - Second noun (optional)
    'question_type',   # 14 - location / appearance / direction / other
    'question',        # 15 - Clarification question
    'answer',          # 16 - Answer to question
    'necessary',       # 17 - Whether QA is necessary for task completion
]

# NOTE: The CSV has a header row — skip it with header=0 and use our own names.
df = pd.read_csv('dialfred_human_qa.csv', header=0, names=column_names, skiprows=1)

print(f"Total rows: {len(df):,}")
print(f"\nSplits: {df['split'].value_counts().to_dict()}")
print(f"\nQuestion types: {df['question_type'].value_counts().to_dict()}")
print(f"\nNecessary distribution: {df['necessary'].value_counts().to_dict()}")
df.head(3)
```

```
Total rows: 53,003

Splits: {'train': 49028, 'valid_seen': 1998, 'valid_unseen': 1977}

Question types: {'location': 30694, 'appearance': 10727, 'direction': 8075, 'other': 3507}

Necessary distribution: {True: 43002, False: 10001}
```

	split	task_id	trial_id	room_type	task_type	subgoal_start	subgo
0	train	look_at_obj_in_light-AlarmClock-None-DeskLamp-316	trial_T20190906_202305_162111	FloorPlan316	look_at_obj_in_light	28.4	
		look_at_obj_in_light-					

Next steps: Generate code with df New interactive sheet

3. Prepare Ambiguity Labels & Clean Data

```
# ── Derive ambiguity label ──
# An instruction is "ambiguous" if the human annotator said clarification was necessary.

df['instruction'] = df['instruction'].astype(str).str.strip()
df['necessary'] = df['necessary'].astype(str).str.strip().str.lower()

# Map to binary: 1 = ambiguous (necessary=true), 0 = not ambiguous
df['ambiguous'] = df['necessary'].map(lambda x: 1 if x in ('true', '1', 'yes') else 0)

# Drop rows with missing instructions
df = df.dropna(subset=['instruction'])
df = df[df['instruction'] != 'nan']

print(f"Rows after cleaning: {len(df):,}")
print(f"Ambiguous (1): {df['ambiguous'].sum():,}")
print(f"Not ambiguous (0): {(df['ambiguous'] == 0).sum():,}")
print(f"Ratio: {df['ambiguous'].mean():.2%} ambiguous")
```

```
Rows after cleaning: 53,003
Ambiguous (1): 43,002
Not ambiguous (0): 10,001
Ratio: 81.13% ambiguous
```

```
# ── Aggregate to instruction level ──
# Multiple annotators may have labeled the same instruction.
#
# Strategy: MAJORITY VOTE — label as ambiguous only if >50% of
# annotators flagged it as necessary. This avoids the problem where
# using max() makes 97% of instructions ambiguous.

instruction_df = df.groupby(['instruction', 'split']).agg(
    ambiguous_mean=('ambiguous', 'mean'),   # fraction who said necessary
    num_annotations=('ambiguous', 'count'),
    question_types=('question_type', lambda x: list(x.unique())),
    task_type=('task_type', 'first'),
    room_type=('room_type', 'first'),
).reset_index()

# Majority vote: ambiguous if >50% of annotators said necessary
instruction_df['ambiguous'] = (instruction_df['ambiguous_mean'] > 0.5).astype(int)

print(f"\nUnique instructions: {len(instruction_df):,}")
print(f"Ambiguous: {instruction_df['ambiguous'].sum():,}")
print(f"Not ambiguous: {(instruction_df['ambiguous'] == 0).sum():,}")
print(f"Ratio: {instruction_df['ambiguous'].mean():.2%} ambiguous")
print(f"\nAnnotator agreement distribution:")
print(instruction_df['ambiguous_mean'].describe())
```

```
Unique instructions: 4,574
Ambiguous: 3,911
Not ambiguous: 663
Ratio: 85.51% ambiguous
```

```
Annotator agreement distribution:
count    4574.000000
mean        0.843312
std         0.230910
min         0.000000
25%         0.750000
50%         1.000000
75%         1.000000
max         1.000000
Name: ambiguous_mean, dtype: float64
```

```
# ── Show some examples ──
print("=" * 70)
print("AMBIGUOUS instructions (label = 1):")
print("=" * 70)
for _, row in instruction_df[instruction_df['ambiguous'] == 1].head(5).iterrows():
    print(f"  → {row['instruction']}")

print()
print("=" * 70)
print("NOT AMBIGUOUS instructions (label = 0):")
print("=" * 70)
for _, row in instruction_df[instruction_df['ambiguous'] == 0].head(5).iterrows():
    print(f"  → {row['instruction']}")
```

```
======================================================================
AMBIGUOUS instructions (label = 1):
======================================================================
  → Chill the apple.
  → Chill the apple.
  → Chill the bowl.
  → Chill the bread.
  → Chill the bread.

======================================================================
NOT AMBIGUOUS instructions (label = 0):
======================================================================
  → Chill the bread.
  → Chill the fridge.
  → Clean the plate.
  → Cook the cup.
  → Cool the tomato.
```

4. Generate Embeddings

⚠️ TEMPORARY: Text-only embeddings

We use `all-MiniLM-L6-v2` (384-dim) as a placeholder encoder.

🔄 FUTURE: Replace with BLIP-2 multimodal embeddings

When the BLIP-2 colab is ready:

1. Extract fused (image + text) embeddings from BLIP-2
2. Save them as a `.pt` or `.npy` file
3. Load them in Section 4b below (uncomment that cell)
4. Update `EMBEDDING_DIM` to match BLIP-2 output dimension

```
# ── 4a. Temporary: sentence-transformer embeddings ──
# This will be replaced by BLIP-2 embeddings in the future.

print("Loading sentence-transformer model...")
st_model = SentenceTransformer('all-MiniLM-L6-v2')

instructions = instruction_df['instruction'].tolist()
labels = instruction_df['ambiguous'].values
splits = instruction_df['split'].values

print(f"Encoding {len(instructions):,} instructions...")
embeddings = st_model.encode(instructions, show_progress_bar=True, batch_size=128)
embeddings = np.array(embeddings)

EMBEDDING_DIM = embeddings.shape[1]
print(f"Embedding shape: {embeddings.shape}  (dim = {EMBEDDING_DIM})")
```

```
Loading sentence-transformer model...
modules.json: 100%  349/349 [00:00<00:00, 19.0kB/s]
config_sentence_transformers.json: 100%  116/116 [00:00<00:00, 9.56kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
README.md: 10.5k/? [00:00<00:00, 895kB/s]
sentence_bert_config.json: 100%  53.0/53.0 [00:00<00:00, 3.24kB/s]
config.json: 100%  612/612 [00:00<00:00, 46.7kB/s]
model.safetensors: 100%  90.9M/90.9M [00:01<00:00, 208MB/s]
Loading weights: 100%  103/103 [00:00<00:00, 399.78it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key                     | Status     |  |
------------------------+------------+--+-
embeddings.position_ids | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100%  350/350 [00:00<00:00, 40.3kB/s]
vocab.txt: 232k/? [00:00<00:00, 10.0MB/s]
tokenizer.json: 466k/? [00:00<00:00, 25.2MB/s]
special_tokens_map.json: 100%  112/112 [00:00<00:00, 10.7kB/s]
config.json: 100%  190/190 [00:00<00:00, 19.5kB/s]
Encoding 4,574 instructions...
Batches: 100%  36/36 [00:01<00:00, 40.71it/s]
Embedding shape: (4574, 384)  (dim = 384)
```

```
# ── 4b. FUTURE: Load BLIP-2 multimodal embeddings ──
# Uncomment and modify this cell when BLIP-2 embeddings are ready.
#
# import numpy as np
#
# # Option A: Load from .npy file
# embeddings = np.load('blip2_embeddings.npy')  # shape: (N, EMBEDDING_DIM)
# labels = np.load('blip2_labels.npy')           # shape: (N,)
#
# # Option B: Load from .pt file (PyTorch tensor)
# # data = torch.load('blip2_embeddings.pt')
# # embeddings = data['embeddings'].numpy()
# # labels = data['labels'].numpy()
#
# EMBEDDING_DIM = embeddings.shape[1]
# print(f"Loaded BLIP-2 embeddings: {embeddings.shape} (dim = {EMBEDDING_DIM})")
```

5. Train / Val / Test Splits

```
# ── Use DialFRED's original splits ──
# train → train
# valid_seen → val
# valid_unseen → test
```

```
train_mask = splits == 'train'
val_mask = splits == 'valid_seen'
test_mask = splits == 'valid_unseen'

# If the split names don't match, fall back to random split
if train_mask.sum() == 0:
    print(f"⚠️ Standard splits not found. Falling back to random 70/15/15 split.")
    print(f"  Unique split values found: {np.unique(splits)}")

    idx_train, idx_temp = train_test_split(
        np.arange(len(embeddings)), test_size=0.3, random_state=42, stratify=labels
    )
    idx_val, idx_test = train_test_split(
        idx_temp, test_size=0.5, random_state=42, stratify=labels[idx_temp]
    )
    train_mask = np.zeros(len(embeddings), dtype=bool)
    val_mask = np.zeros(len(embeddings), dtype=bool)
    test_mask = np.zeros(len(embeddings), dtype=bool)
    train_mask[idx_train] = True
    val_mask[idx_val] = True
    test_mask[idx_test] = True

X_train, y_train = embeddings[train_mask], labels[train_mask]
X_val, y_val = embeddings[val_mask], labels[val_mask]
X_test, y_test = embeddings[test_mask], labels[test_mask]

print(f"Train: {len(X_train):,} ({y_train.mean():.1%} ambiguous)")
print(f"Val:   {len(X_val):,} ({y_val.mean():.1%} ambiguous)")
print(f"Test:  {len(X_test):,} ({y_test.mean():.1%} ambiguous)")
```

```
Train: 3,546 (86.3% ambiguous)
Val:   542 (80.4% ambiguous)
Test:  486 (85.4% ambiguous)
```

6. PyTorch Dataset & DataLoader

```
class AmbiguityDataset(Dataset):
    """Dataset for ambiguity classification.

    Takes pre-computed embeddings and binary labels.
    Works with any embedding source (sentence-transformer, BLIP-2, etc.).
    """

    def __init__(self, embeddings, labels):
        self.embeddings = torch.tensor(embeddings, dtype=torch.float32)
        self.labels = torch.tensor(labels, dtype=torch.float32)

    def __len__(self):
        return len(self.labels)

    def __getitem__(self, idx):
        return self.embeddings[idx], self.labels[idx]


BATCH_SIZE = 64

train_dataset = AmbiguityDataset(X_train, y_train)
val_dataset = AmbiguityDataset(X_val, y_val)
test_dataset = AmbiguityDataset(X_test, y_test)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=False)

print(f"Train batches: {len(train_loader)}")
print(f"Val batches:   {len(val_loader)}")
print(f"Test batches:  {len(test_loader)}")
```

```
Train batches: 56
Val batches:   9
Test batches:  8
```

7. MLP Classifier Architecture

```
class AmbiguityMLP(nn.Module):
    """MLP head for binary ambiguity classification.

    Architecture:
        embedding → FC → ReLU → Dropout → FC → ReLU → Dropout → FC → sigmoid

    Args:
        input_dim:  Dimension of the input embedding.
                    - 384 for all-MiniLM-L6-v2 (current temporary encoder)
                    - 768 for BLIP-2 default (future)
                    - 1408 for BLIP-2 Q-Former output (future)
        hidden_dim: Hidden layer size (default 256)
        dropout:    Dropout rate (default 0.3)
        num_classes: 1 for binary classification (default).
                     Change to N for multi-class ambiguity types in the future.
    """

    def __init__(self, input_dim, hidden_dim=256, dropout=0.3, num_classes=1):
        super().__init__()
        self.num_classes = num_classes

        self.network = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim),
            nn.Dropout(dropout),

            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.ReLU(),
            nn.BatchNorm1d(hidden_dim // 2),
            nn.Dropout(dropout),

            nn.Linear(hidden_dim // 2, num_classes),
        )

    def forward(self, x):
        logits = self.network(x)
        if self.num_classes == 1:
            return logits.squeeze(-1)  # (batch,) for binary
        return logits  # (batch, num_classes) for multi-class


# ── Instantiate ──
model = AmbiguityMLP(
    input_dim=EMBEDDING_DIM,
    hidden_dim=256,
    dropout=0.3,
    num_classes=1,  # Binary for now. Change to N for multi-class.
).to(device)

print(model)
total_params = sum(p.numel() for p in model.parameters())
print(f"\nTotal parameters: {total_params:,}")
```

```
AmbiguityMLP(
  (network): Sequential(
    (0): Linear(in_features=384, out_features=256, bias=True)
    (1): ReLU()
    (2): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=256, out_features=128, bias=True)
    (5): ReLU()
    (6): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (7): Dropout(p=0.3, inplace=False)
    (8): Linear(in_features=128, out_features=1, bias=True)
  )
)

Total parameters: 132,353
```

8. Training Setup

```
# ── Handle class imbalance ──
# Compute positive weight for BCEWithLogitsLoss so the model pays
# more attention to the minority class.
# pos_weight > 1 → upweights positives (when positives are rare)
# pos_weight < 1 → downweights positives (when positives are majority)

num_pos = int(y_train.sum())
num_neg = len(y_train) - num_pos
pos_weight = torch.tensor([num_neg / max(num_pos, 1)], dtype=torch.float32).to(device)
print(f"Class balance — Pos: {num_pos:,}  Neg: {num_neg:,}  pos_weight: {pos_weight.item():.2f}")

# If the classes are very imbalanced (>90% one class), warn the user
majority_ratio = max(num_pos, num_neg) / (num_pos + num_neg)
if majority_ratio > 0.9:
    print(f"⚠️ WARNING: {majority_ratio:.0%} of samples belong to one class.")
    print(f"   Consider adjusting the labeling strategy if results are poor.")

# ── Loss & Optimizer ──
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight)
optimizer = optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='max', factor=0.5, patience=5
)

NUM_EPOCHS = 50
EARLY_STOP_PATIENCE = 10
```

```
Class balance — Pos: 3,060  Neg: 486  pos_weight: 0.16
```

9. Training Loop

```
def evaluate(model, loader, criterion, device):
    """Evaluate model on a DataLoader. Returns loss, accuracy, F1, preds, labels."""
    model.eval()
    all_preds, all_labels = [], []
    total_loss = 0.0

    with torch.no_grad():
        for embeddings, labels in loader:
            embeddings, labels = embeddings.to(device), labels.to(device)
            logits = model(embeddings)
            loss = criterion(logits, labels)
            total_loss += loss.item() * len(labels)

            preds = (torch.sigmoid(logits) >= 0.5).long()
            all_preds.extend(preds.cpu().numpy())
            all_labels.extend(labels.cpu().numpy())

    all_preds = np.array(all_preds)
    all_labels = np.array(all_labels)
    avg_loss = total_loss / len(all_labels)
    acc = accuracy_score(all_labels, all_preds)
    f1 = f1_score(all_labels, all_preds, zero_division=0)

    return avg_loss, acc, f1, all_preds, all_labels
```

```
# ── Training ──
history = {'train_loss': [], 'val_loss': [], 'train_f1': [], 'val_f1': [], 'val_acc': []}
best_val_f1 = 0.0
patience_counter = 0

for epoch in range(1, NUM_EPOCHS + 1):
    # — Train —
    model.train()
    train_loss_sum = 0.0
    train_preds_all, train_labels_all = [], []

    for embeddings, labels in train_loader:
        embeddings, labels = embeddings.to(device), labels.to(device)

        optimizer.zero_grad()
        logits = model(embeddings)
        loss = criterion(logits, labels)
        loss.backward()
```

```
        optimizer.step()

        train_loss_sum += loss.item() * len(labels)
        preds = (torch.sigmoid(logits) >= 0.5).long()
        train_preds_all.extend(preds.cpu().numpy())
        train_labels_all.extend(labels.cpu().numpy())

    train_loss = train_loss_sum / len(train_preds_all)
    train_f1 = f1_score(train_labels_all, train_preds_all, zero_division=0)

    # — Validate —
    val_loss, val_acc, val_f1, _, _ = evaluate(model, val_loader, criterion, device)

    # — LR scheduler —
    scheduler.step(val_f1)

    # — Log —
    history['train_loss'].append(train_loss)
    history['val_loss'].append(val_loss)
    history['train_f1'].append(train_f1)
    history['val_f1'].append(val_f1)
    history['val_acc'].append(val_acc)

    if epoch % 5 == 0 or epoch == 1:
        print(
            f"Epoch {epoch:3d}/{NUM_EPOCHS} │ "
            f"Train Loss: {train_loss:.4f}  F1: {train_f1:.4f} │ "
            f"Val Loss: {val_loss:.4f}  Acc: {val_acc:.4f}  F1: {val_f1:.4f}"
        )

    # — Early stopping & checkpoint —
    if val_f1 > best_val_f1:
        best_val_f1 = val_f1
        patience_counter = 0
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'val_f1': val_f1,
            'val_acc': val_acc,
            'embedding_dim': EMBEDDING_DIM,
        }, 'best_ambiguity_mlp.pt')
    else:
        patience_counter += 1
        if patience_counter >= EARLY_STOP_PATIENCE:
            print(f"\n■ Early stopping at epoch {epoch} (no improvement for {EARLY_STOP_PATIENCE} epochs)")
            break

print(f"\n✅ Best Val F1: {best_val_f1:.4f}")
```

```
Epoch   1/50 │ Train Loss: 0.1935  F1: 0.6309 │ Val Loss: 0.2378  Acc: 0.6900  F1: 0.8078
Epoch   5/50 │ Train Loss: 0.1595  F1: 0.7214 │ Val Loss: 0.3205  Acc: 0.7048  F1: 0.8122
Epoch  10/50 │ Train Loss: 0.1440  F1: 0.7804 │ Val Loss: 0.4828  Acc: 0.7417  F1: 0.8451
Epoch  15/50 │ Train Loss: 0.1356  F1: 0.8221 │ Val Loss: 0.5170  Acc: 0.6993  F1: 0.8137
Epoch  20/50 │ Train Loss: 0.1173  F1: 0.8390 │ Val Loss: 0.6793  Acc: 0.7343  F1: 0.8407

■ Early stopping at epoch 20 (no improvement for 10 epochs)

✅ Best Val F1: 0.8451
```

10. Training Curves

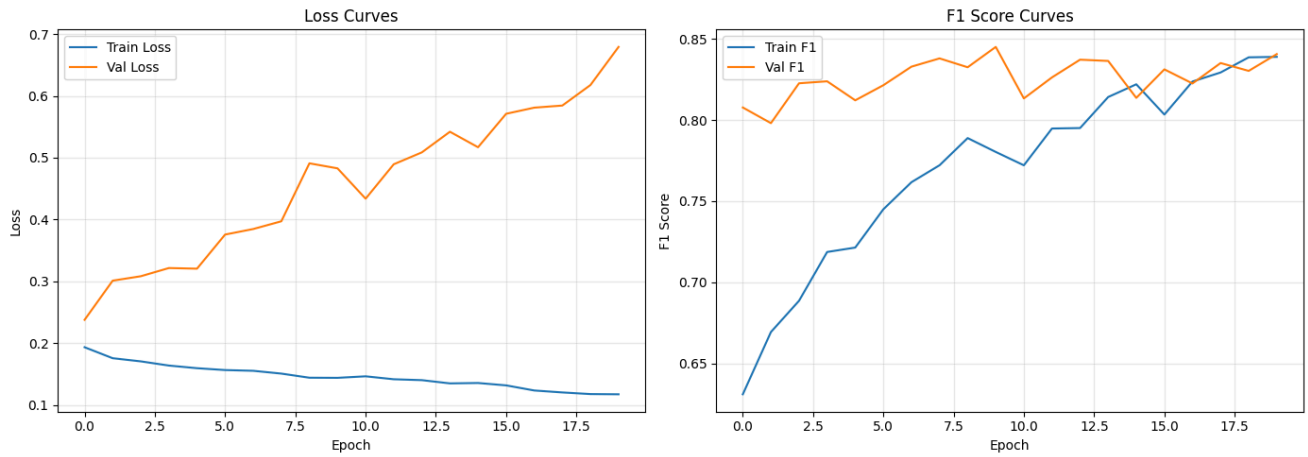
```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Loss
axes[0].plot(history['train_loss'], label='Train Loss')
axes[0].plot(history['val_loss'], label='Val Loss')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Loss')
axes[0].set_title('Loss Curves')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# F1
axes[1].plot(history['train_f1'], label='Train F1')
```

```
axes[1].plot(history['val_f1'], label='Val F1')
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('F1 Score')
axes[1].set_title('F1 Score Curves')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('training_curves.png', dpi=150, bbox_inches='tight')
plt.show()
```



11. Test Evaluation

```
# ── Load best checkpoint ──
checkpoint = torch.load('best_ambiguity_mlp.pt', map_location=device)
model.load_state_dict(checkpoint['model_state_dict'])
print(f"Loaded best model from epoch {checkpoint['epoch']} (Val F1: {checkpoint['val_f1']:.4f})")

# ── Evaluate on test set ──
test_loss, test_acc, test_f1, test_preds, test_labels = evaluate(
    model, test_loader, criterion, device
)

test_precision = precision_score(test_labels, test_preds, zero_division=0)
test_recall = recall_score(test_labels, test_preds, zero_division=0)

print(f"\n{'='*50}")
print(f"  TEST RESULTS (valid_unseen)")
print(f"{'='*50}")
print(f"  Accuracy:  {test_acc:.4f}")
print(f"  F1 Score:  {test_f1:.4f}")
print(f"  Precision: {test_precision:.4f}")
print(f"  Recall:    {test_recall:.4f}")
print(f"  Loss:      {test_loss:.4f}")
print(f"{'='*50}")
```

```
Loaded best model from epoch 10 (Val F1: 0.8451)

==================================================
  TEST RESULTS (valid_unseen)
==================================================
  Accuracy:  0.7881
  F1 Score:  0.8778
  Precision: 0.8645
  Recall:    0.8916
  Loss:      0.3300
==================================================
```

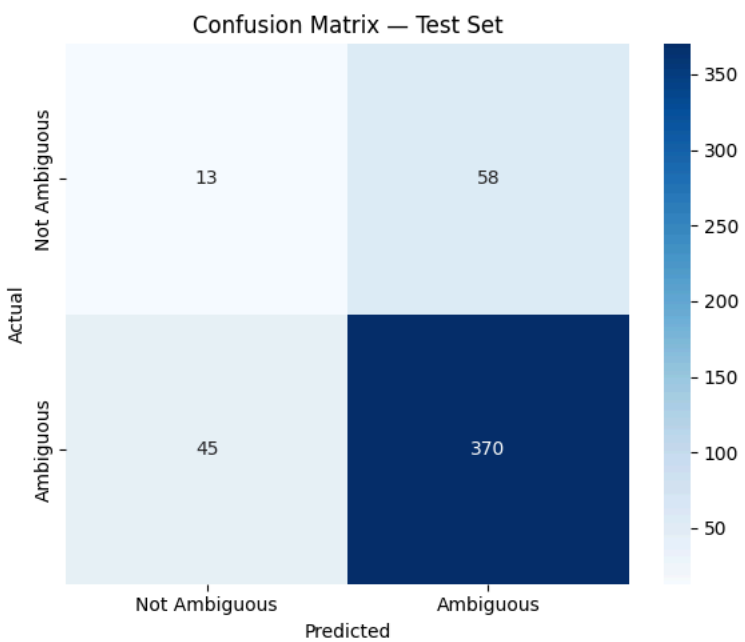
```
# ── Classification Report ──
print("\nClassification Report:")
print(classification_report(
    test_labels, test_preds,
    target_names=['Not Ambiguous', 'Ambiguous']
))
```

```
Classification Report:
               precision    recall  f1-score   support

Not Ambiguous       0.22      0.18      0.20        71
    Ambiguous       0.86      0.89      0.88       415

     accuracy                           0.79       486
    macro avg       0.54      0.54      0.54       486
 weighted avg       0.77      0.79      0.78       486
```

```
# ── Confusion Matrix ──
cm = confusion_matrix(test_labels, test_preds)
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(
    cm, annot=True, fmt='d', cmap='Blues',
    xticklabels=['Not Ambiguous', 'Ambiguous'],
    yticklabels=['Not Ambiguous', 'Ambiguous'],
    ax=ax
)
ax.set_xlabel('Predicted')
ax.set_ylabel('Actual')
ax.set_title('Confusion Matrix — Test Set')
plt.tight_layout()
plt.savefig('confusion_matrix.png', dpi=150, bbox_inches='tight')
plt.show()
```



12. Inference Helper

Quick function to classify new instructions.

```
def predict_ambiguity(instruction: str, model, encoder, device, threshold=0.5):
    """
    Predict whether an instruction is ambiguous.

    Args:
        instruction: Natural language instruction string.
        model:       Trained AmbiguityMLP.
        encoder:     Embedding model (sentence-transformer now, BLIP-2 later).
                     Must have an .encode() method.
        device:      torch device
```

```
        device:      torch device.
        threshold:   Classification threshold (default 0.5).

    Returns:
        dict with 'label', 'confidence', 'instruction'
    """
    model.eval()
    emb = encoder.encode([instruction])
    emb_tensor = torch.tensor(emb, dtype=torch.float32).to(device)

    with torch.no_grad():
        logit = model(emb_tensor)
        prob = torch.sigmoid(logit).item()

    label = 'Ambiguous' if prob >= threshold else 'Not Ambiguous'
    return {
        'instruction': instruction,
        'label': label,
        'confidence': prob if prob >= threshold else 1 - prob,
    }


# ── Test it ──
test_instructions = [
    "Move that away",
    "Pick up the red mug from the counter and place it in the sink",
    "Go over there",
    "Put the knife on the table next to the toaster",
    "Clean it",
    "Turn right and walk to the fridge",
]

print(f"{'Instruction':<60} {'Prediction':<20} {'Confidence'}")
print("—" * 95)
for instr in test_instructions:
    result = predict_ambiguity(instr, model, st_model, device)
    print(f"{result['instruction']:<60} {result['label']:<20} {result['confidence']:.3f}")
```

```
Instruction                                                  Prediction           Confidence
———————————————————————————————————————————————————————————————————————————————————————————————
Move that away                                               Ambiguous            0.998
Pick up the red mug from the counter and place it in the sink Ambiguous            0.508
Go over there                                                Ambiguous            0.993
Put the knife on the table next to the toaster               Ambiguous            0.502
Clean it                                                     Ambiguous            0.997
Turn right and walk to the fridge                            Ambiguous            0.567
```

Future Work / TODOs

Plug in BLIP-2 embeddings

1. In the BLIP-2 colab, extract fused multimodal embeddings for each (image, instruction) pair
2. Save as `blip2_embeddings.npy` and `blip2_labels.npy`
3. Uncomment **Section 4b** and comment out **Section 4a**
4. Update `EMBEDDING_DIM` (likely 768 or 1408)
5. Re-run from Section 5 onward

Multi-class ambiguity classification

When ready to classify ambiguity *types* (location, appearance, direction, other):

1. Change `num_classes=1` → `num_classes=4` in the `AmbiguityMLP` constructor
2. Switch `BCEWithLogitsLoss` → `CrossEntropyLoss`
3. Labels become integers 0–3 instead of binary